Dijkstra's Shortest Path Algorithm

Computer Science Core Concepts & Algorithmic Foundations

1. The Core Concept (Made Simple)

Imagine you are planning a road trip from City **A** to City **E** across a map filled with multiple interconnected routes and toll fees. You want to find the absolute cheapest or shortest path possible without guessing.

Dijkstra's Algorithm works like a expanding wave of water. Starting at your origin node, water spreads outwards along every available path at a uniform rate. The very first moment water touches a town, that arrival route is guaranteed to be the shortest possible path to that town.

> **Key Principle: Greedy Strategy & Path Relaxation**
> Dijkstra's algorithm is a **greedy algorithm**. At each step, it visits the unvisited location with the smallest currently known tentative distance. When visiting a node, it updates (relaxes) the estimated distances to all of its unvisited neighbors.

2. Essential Terminology & Components

Graph Structure (V, E)

Consists of **Vertices (Nodes)** representing intersections or points, and **Edges (Links)** connecting them with specific numerical **Weights** (e.g., distance, cost, time).

Tentative Distance Array

A ledger tracking the shortest known cost from the start node to every other node. Initially set to **0** for the start node and **Infinity (∞)** for all others.

Visited Set

A list of completed nodes whose shortest path from the origin has been finalized. Once added to the visited set, a node is never re-evaluated.

Path Relaxation Operation

The core update formula: If `Distance(A) + Weight(A → B) < Distance(B)`, then update `Distance(B)` to the lower value.

3. Graph Visualization & Setup



4. Step-by-Step Execution Trace

Let us trace the algorithm on the sample graph above, finding the shortest path from start node **A** to destination node **Z**.

Step	Current Node	Distance Updates (Relaxation)	Tentative Distances [A, B, C, D, E, Z]	Visited Set
Init	—	Set Start A=0, all other nodes = ∞	[0, ∞, ∞, ∞, ∞, ∞]	{ }
1	**A**	Relax B: min(∞, 0+4)=4 Relax C: min(∞, 0+2)=2	[0, 4, 2, ∞, ∞, ∞]	{ A }
2	**C**	Relax B: min(4, 2+1)=3 (updated!) Relax E: min(∞, 2+8)=10	[0, 3, 2, ∞, 10, ∞]	{ A, C }
3	**B**	Relax D: min(∞, 3+5)=8	[0, 3, 2, 8, 10, ∞]	{ A, C, B }
4	**D**	Relax Z: min(∞, 8+3)=11	[0, 3, 2, 8, 10, 11]	{ A, C, B, D }
5	**E**	Relax Z: min(11, 10+6)=11 (no change)	[0, 3, 2, 8, 10, 11]	{ A, C, B, D, E }
6	**Z**	Target reached with minimal distance **11**	[0, 3, 2, 8, 10, 11]	{ A, C, B, D, E, Z }

Final Shortest Path: Reconstructing backwards yields A → C → B → D → Z with a total distance of **11**.

Critical Limitation: Negative Edge Weights

Dijkstra's algorithm **fails on graphs with negative edge weights**. Because it assumes adding an edge only increases (or maintains) total distance, once a node is visited, its distance is finalized. If negative weights exist, use the **Bellman-Ford algorithm** instead.

5. High-Level Pseudocode

```
function Dijkstra(Graph, source): create distance array dist[] initialized to Infinity create priority queue PQ dist[source] = 0 PQ.insert(source, 0) while PQ is not empty: u = PQ.extract_min() for each neighbor v of u: weight = edge(u, v) if dist[u] + weight < dist[v]: dist[v] = dist[u] + weight PQ.decrease_key(v, dist[v]) return dist
```

6. Time & Space Complexity Analysis

The performance of Dijkstra's algorithm depends significantly on the data structure used to implement the priority queue for picking the minimum distance unvisited node.

Implementation / Data Structure	Time Complexity	Space Complexity	Best Used For
Array / List Search	$O(V^2)$	$O(V)$	Dense graphs with many edges ($E \approx V^2$)
Binary Min-Heap	$O((V + E) \log V)$	$O(V)$	Sparse graphs, general real-world applications
Fibonacci Heap	$O(E + V \log V)$	$O(V)$	Theoretical optimal efficiency for large sparse networks

7. Real-World Applications

GPS Navigation & Mapping

Services like Google Maps and Apple Maps utilize modified variants of Dijkstra (e.g., A* Search) to compute turn-by-turn routes avoiding traffic and long distances.

Network Routing Protocols

Link-state routing protocols like **OSPF (Open Shortest Path First)** use Dijkstra's algorithm to calculate optimal data packet paths across internet routers.

Flight & Logistics Scheduling

Airlines calculate minimum cost or shortest duration connecting routes across flight networks for passengers and freight delivery.

Social Network Distance

Platforms compute degrees of separation and shortest connection chains between user profiles across social graphs.

8. Summary Comparison

Dijkstra vs. Other Pathfinding Algorithms

- **BFS (Breadth-First Search):** Finds shortest paths only when all edge weights are equal to 1.
- **Dijkstra:** Handles non-negative weighted edges efficiently.
- **A* Search:** Enhances Dijkstra by adding heuristics to guide the search towards a target faster.
- **Bellman-Ford:** Slower than Dijkstra, but supports negative edge weights safely.